
Prompt-Based Collaborative Agent Framework (PCAF) for Enhanced LLM Mathematical Reasoning

David S. Hippocampus*

Department of Computer Science
Cranberry-Lemon University
Pittsburgh, PA 15213
hippo@cs.cranberry-lemon.edu

Abstract

The inherent unreliability of Large Language Models (LLMs) in complex, multi-step calculations necessitates architectural interventions to achieve trustworthy accuracy in mathematical reasoning. This paper introduces the Prompt-Based Collaborative Agent Framework (PCAF), a novel, architecture-driven approach aimed at significantly improving final-answer accuracy on challenging benchmarks such as MathArena (including AIME 2025 and HMMT Feb 2025 subsets). The method utilizes a single LLM instance (DeepSeek-V3-03-24) and employs sequential, role-specific prompting to simulate a sophisticated, three-agent collaborative system. The framework operates in a continuous, closed-loop refinement cycle defined by three specialized agents: a Solver, which generates the solution under a strict Dual-Method Verification and Code-First Mandate; a Verifier, which acts as a Skeptical Math Auditor to generate a structured JSON critique; and a Planner, which translates the critique into an explicit, non-negotiable MANDATORY ACTION instruction for the next attempt. This architecture transforms the LLM’s linear reasoning into a persistent self-correction mechanism. Through systematic evaluation, we establish a robust baseline using Zero-Shot Chain-of-Thought (CoT) and demonstrate that the PCAF’s structured intervention yields a significant uplift in final accuracy in comparison to the baseline (51.3% versus 39.6%), with a 37.4% recovery rate confirming the efficacy of deterministic, architecture-driven reflection over traditional self-correction techniques.

1 Introduction

The pursuit of reliable mathematical reasoning in Large Language Models (LLMs) is hindered by two persistent challenges: computational imprecision and stubbornness in self-correction. While LLMs excel at generating coherent solution narratives (Chain-of-Thought), they frequently fail at the final, precise steps—arithmetic, algebra, and complex counting—which leads to high rates of final-answer error on benchmarks like MathArena. Furthermore, models typically struggle to effectively diagnose and pivot from their initial logical errors, making standard self-correction techniques unreliable.

To overcome these limitations, this project implements the Prompt-Based Collaborative Agent Framework (PCAF). Instead of relying on expensive model fine-tuning, PCAF utilizes an architecture-driven reasoning approach. It simulates a multi-agent system within a single LLM instance through sequential, role-specific prompting. The framework dynamically cycles the LLM through three specialized roles—Solver, Verifier, and Planner—in an iterative, closed-loop refinement process. The

*Use footnote for providing further information about author (webpage, alternative address)—*not* for acknowledging funding agencies.

core objective is to demonstrate that this structured collaborative architecture significantly increases the final-answer accuracy and trustworthiness compared to the model’s standard Zero-Shot baseline.

The key contributions of this work are:

- **Novel Algorithmic Structure:** We introduce the PCAF loop, which formalizes the iterative roles of Generation (Solver), Critique (Verifier), and Coordination (Planner), forcing systematic logical scrutiny.
- **Enforced Grounding:** We implement the mandatory integration of a persistent Python sandbox environment ("Code" tool) to enforce Dual-Method Verification (Analytical vs. Naive Brute Force), ensuring mathematical conclusions are derived from verifiable, executed code.
- **Targeted Improvement:** We apply and evaluate PCAF on the challenging final-answer sub-sections of the MathArena benchmark (AIME 2025, HMMT Feb 2025 Final-Answer Competitions) to provide a clear, quantitative measure of accuracy uplift.

2 Related Work

The Prompt-Based Collaborative Agent Framework (PCAF) is informed by two established and growing areas of research in LLMs: Reflective Reasoning and Verification and Multi-Agent Systems for Problem Solving.

2.1 Reflective Reasoning and Verification

Early methods in mathematical reasoning focused on generating a single, detailed Chain-of-Thought (CoT). However, recent work emphasizes enhancing reliability through self-critique. Approaches such as those presented by Wang et al. (2025) [3], who introduce a collaboration mechanism for long CoT reasoning, and Li et al. (2025) [1], who explores LLM agents for automated proof generation using knowledge graphs, highlight the value of structured introspection. These systems often succeed in generating multiple solution paths or utilizing formal methods to check internal consistency. However, a common limitation is the nature of the corrective feedback; the system may identify an error but lack a dedicated, high-level mechanism to effectively re-plan the solution strategy. The feedback loop often remains constrained to simple re-prompts or voting systems.

2.2 Multi-Agent Systems and Formal Verification

A parallel line of inquiry involves distributing tasks among distinct, specialized LLM instances to simulate collaborative environments. Works like Varambally et al. (2025) [2], which describe a system for recursively building formal proofs, employ specialized agents (a "formal prover" and a "critic") to ensure soundness. This multi-model approach is computationally powerful and minimizes agent bias but requires the coordination and hosting of multiple distinct models. Furthermore, when these systems interact with external tools, such as the formal prover used, the communication requires complex integration layers.

3 Methodology

3.1 Framework Overview

Describe the iterative loop ($N = 3$ max iterations). The PCAF is a deterministic, iterative, and closed-loop system designed to enforce self-correction and dual-verification. The process is initiated with the problem statement and runs for a maximum of $N = 4$ total attempts (one initial attempt plus three retries). The core cycle involves three sequential steps:

1. **Attempt (Solver):** The Solver generates a solution and its associated execution trace.
2. **Verification (Verifier):** The Verifier audits the complete trace against a structured checklist and returns a structured critique.
3. **Correction/Re-plan (Planner $\rightarrow$ Solver):** If the Verifier rejects the solution, the Planner translates the critique into an explicit, targeted instruction that overrides the Solver’s default behavior, forcing a

specific corrective action in the next attempt. The loop terminates successfully if the Verifier accepts the solution, or unsuccessfully if the maximum number of attempts is reached.

3.2 The Agents (Cite specific prompts from your code):

The framework’s robustness derives from the dynamic specialization of the single LLM into three distinct agents via role-specific system prompts.

3.2.1 The Solver

The Solver is the generative component, constrained by a strict system prompt `get_solver_prompt` that enforces the Dual-Verification Protocol for grounding and reliability.

“Two Strictly Different Methods” Mandate: The Solver must use two conceptually distinct approaches to arrive at the same answer:

- **Analytical Method (Method 1):** Utilizes formal mathematics, algebra, or efficient libraries (e.g., `sympy`).
- **Naive Brute-Force (Method 2):** Requires a simple, unoptimized Python script to simulate or iterate through all possibilities.

“Code-First” Mandate: The Solver is required to execute the code for both methods and explicitly compare the outputs, ensuring the final answer is a value confirmed by an external tool. This process is crucial for preventing numerical hallucinations.

3.2.2 The Verifier

The Verifier’s role is that of a Skeptical Math Auditor (`VERIFIER_ROLE`). It receives the Solver’s full reasoning and code outputs and audits the solution based on a specific **Audit Checklist** (e.g., consistency between text and code, presence of the dual-method check, and logic trap checks).

The Verifier is strictly constrained to output a structured JSON critique that adheres to a predefined schema:

```
{JSON Schema:  "valid":  bool, "error_type":  str, "critique":  
str }
```

The `error_type` field serves as the primary, discrete signal for the Planner, classifying failures into three key categories:

LOGIC_FLAW A fundamental error in the mathematical approach or application of a principle.

METHOD_CONFLICT The outputs of the two mandated methods (Analytical vs. Naive) did not match, or the comparison step was skipped.

VALUE_MISMATCH The logic or code was mostly correct, but the final textual answer did not match the computed output (e.g., failed to apply a final transformation like modulo or rounding).

3.2.3 The Planner

The Planner is not a separate LLM call but rather a deterministic, heuristic-driven function (`construct_planner_feedback`) that translates the Verifier’s JSON rejection into a specific, high-leverage instruction for the subsequent Solver attempt.

The Planner’s logic ensures that corrections are targeted and strategy-shifting:

Targeted Reset for VALUE_MISMATCH If the `VALUE_MISMATCH` flag is raised, the Planner generates an instruction to “Wipe memory of the ‘known’ answer and re-derive it from scratch using code.” This counteracts the LLM’s tendency to stubbornly repeat a flawed answer despite being shown the correct number in the trace.

Strategy Shift for CONCEPTUAL_FLAW For a conceptual failure in geometric problems, the Planner mandates a shift, e.g., “Switch to Coordinate Geometry. Place a vertex at (0,0) and use the Shoelace Formula.”

This architecture ensures the correction loop is not a generic “try again,” but a precise, planned intervention.

3.3 Tool Use

The execution of mathematical code is handled by a custom, stateful environment, the `PersistentSolverSandbox`. This tool is essential for providing verifiable grounding:

Persistent Environment: Unlike typical stateless code execution tools, the sandbox maintains its global state (`self.globals`) across consecutive code blocks within a single Solver attempt. This enables complex, multi-block workflows in which a function is defined in one block and then invoked or modified in subsequent blocks.

Comprehensive Imports: The sandbox is pre-loaded with essential mathematical and symbolic libraries (`math`, `sympy`, `numpy`) to support both analytical and naive solution methods.

Robust I/O Capture: The sandbox reliably captures all standard output (`stdout`) and handles common runtime issues (e.g., infinite loops via timeouts, `IndentationError`, and `SyntaxError`), returning any error messages to the Solver as an `OBSERVATION`.

This feedback loop is critical, as it allows the Solver to debug its own code syntax and execution failures.

The Solver: Reference `get_solver_prompt`. Explain the "Two Strictly Different Methods" mandate (Analytical vs. Naive Brute-Force) and the "Code-First Mandate".

The Verifier: Reference `VERIFIER_ROLE` and `VERIFIER_JSON_SCHEMA`. Explain how it acts as a "Skeptical Math Auditor" looking for specific error categories (`LOGIC_FLAW`, `METHOD_CONFLICT`, `VALUE_MISMATCH`).

The Planner: Reference `construct_planner_feedback`. Explain the deterministic routing logic. Highlight how it translates a JSON error into a specific instruction (e.g., “If `VALUE_MISMATCH` -> Wipe memory, re-calculate”).

Reference `PersistentSolverSandbox`. Explain how you use a persistent Python environment to maintain state variables between code blocks, handling imports (`sympy`, `numpy`) and capturing `stdout`.

4 Experiments

To evaluate the efficacy of the Prompt-Based Collaborative Agent Framework, we benchmarked the system on a subset of high-difficulty problems from the MathArena benchmark (AIME 2025 and HMMT February 2025). These datasets were selected for their complexity, often requiring multi-step reasoning that challenges standard Zero-Shot Chain-of-Thought approaches as well as a comparison to the baseline.

4.1 Experimental Setup

Implementation. The framework utilizes the DeepSeek-V3-0324 model hosted on Azure OpenAI. The architecture consists of three specialized agents—Solver, Verifier, and Planner—operating within a single LLM context. A key component is the Solver’s `PersistentSolverSandbox`, which maintains variable state across turns to allow for iterative code refinement.

Evaluation Protocol. To account for the stochastic nature of LLMs, we adopted a robust evaluation strategy. For each problem in the test set, we performed $K = 4$ independent runs of the full PCAF loop. The reported accuracy for a problem is the average success rate across these runs ($Score \in [0.0, 1.0]$).

Metrics. We assessed performance using three metrics:

- **Accuracy (Pass@K):** The average correctness of the final integer answer across the 4 independent runs.
- **Resilience (Recovery Rate):** The percentage of correct runs where the system initially failed (Turn 1) but successfully self-corrected in subsequent turns (Turn 2–4).

- **Baseline Comparison:** Performance is compared against a Zero-Shot CoT baseline using the same underlying model.

5 Results

Our experiments on the MathArena benchmark demonstrate that the PCAF architecture yields a quantifiable improvement in reliability compared to the standard one-shot approach defined in the benchmark’s evaluation protocol [4]. By aggregating performance across multiple independent runs, we observe that the system’s collaborative loop effectively converts initial reasoning errors into successful solutions.

5.1 Quantitative Performance

Table 1 summarizes the performance metrics on the AIME 2025 and HMMT February 2025 datasets. We report the **Pass@4** accuracy (average score across 4 independent runs per problem) to account for generation variance.

Table 1: Performance of PCAF on MathArena Subsets. **Recovery Rate** denotes the percentage of correct solutions that required self-correction (Turns > 1).

Dataset	PCAF Accuracy	Avg. Turns	Recovery Rate	Baseline
AIME 2025 (Subset)	56.7%	2.16	38.2%	50.0%
HMMT Feb ’25 (Subset)	45.8%	2.47	36.6%	29.17%

The data highlights a significant "Verification Cost": on average, the system required 2.31 turns to reach a final answer. This latency confirms that the default Zero-Shot reasoning (Turn 1) is frequently insufficient for these high-complexity problems, necessitating the intervention of the Verifier and Planner.

5.2 Resilience Analysis

A critical contribution of PCAF is its resilience. We define the *Recovery Rate* as the proportion of successful runs where the system initially failed (Turn 1) but successfully self-corrected in subsequent turns.

- **One-Shot Success (Turn 1):** 62.6% of correct answers were achieved immediately, typically on problems where the analytical derivation was flawless.
- **Collaborative Recovery (Turns 2-4):** 37.4% of correct answers were "rescued" by the agent loop. Without the PCAF architecture, these instances would have been failures.

This 37.4% recovery rate empirically validates the utility of the Planner’s deterministic routing logic.

5.3 Case Study: The “Double-Check” Effect

We highlight a representative recovery trace from the HMMT dataset (Problem ID 1: Sum of divisors of $9!$ ending in 1) to illustrate the framework’s mechanics.

- **Initial Failure (Solver):** The Solver analytically derived divisors $\{1, 81\}$ (Sum: 82), missing the combination $3 \times 7 = 21$.
- **Conflict Detection (Verifier):** The Verifier’s dual-method check flagged a `METHOD_CONFLICT`: “Text derivation (82) does not match Code Output (103).”
- **Correction (Planner):** The Planner issued a mandatory instruction: “*CRITICAL CONFLICT... Ignore previous text... Re-derive using code evidence.*”
- **Resolution (Turn 2):** The Solver synthesized the code output $\{1, 21, 81\}$, corrected its prime factorization logic, and returned the correct answer of 103.

6 Discussion

The results validate our hypothesis that separating the roles of generation (Solver) and auditing (Verifier) significantly enhances reliability in mathematical reasoning.

Efficacy of the “Constitution of Verification”. The recovery rate of 37.4% indicates that the rigid “Constitution”—specifically the requirement that code output must match natural language reasoning—serves as an effective guardrail. In the HMMT case study, a standard Chain-of-Thought model would have hallucinated the answer 82 without a mechanism to flag the missing factor. The `PersistentSolverSandbox` acted as an adversarial truth-grounding mechanism, forcing the model to confront the reality of its calculation errors.

The Planner’s Deterministic Routing. Standard self-correction often fails because LLMs are “stubborn,” frequently repeating their initial error. The PCAF Planner overcomes this by translating vague Verifier critiques into mandatory, strategy-shifting instructions. The high success rate on AIME Number Theory tasks (where the Planner often forced a “Brute Force” check) suggests that deterministic strategy switching is more effective than open-ended “try again” prompting.

Limitations in Geometry. While the system excelled in Algebra and Number Theory (56.7% on AIME), qualitative analysis of failures reveals a weakness in Geometry. When the Solver attempted to parameterize complex shapes (e.g., AIME Problem 2) in Python, it often struggled to translate visual constraints into correct coordinate code. The Verifier correctly flagged inconsistencies, but the Planner lacked a visual modality to offer spatial correction, leading to loop exhaustion. This suggests that for geometric reasoning, the text-based code sandbox must be augmented with Vision-Language Models (VLMs).

7 Conclusion

We introduced the Prompt-Based Collaborative Agent Framework (PCAF), a modular architecture that transforms a single LLM into a self-correcting reasoning system. By orchestrating a Solver, Verifier, and Planner, we achieved a global accuracy of 51.3% on difficult MathArena subsets, recovering over one-third of solutions that would have otherwise failed. Our findings confirm that architecture-driven reasoning—specifically the integration of deterministic planning and code-based verification—offers a viable path toward trustworthy mathematical AI.

References

- [1] Vincent Li, Yule Fu, Tim Knappe, Kevin Han, and Kevin Zhu. Automating mathematical proof generation using large language model agents and knowledge graphs, 2025. *arXiv preprint*.
- [2] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning, 2025. *arXiv preprint*.
- [3] Ruida Wang, Rui Pan, Yuxin Li, Jipeng Zhang, Yizhen Jia, Shizhe Diao, Renjie Pi, Junjie Hu, and Tong Zhang. Ma-lot: Model-collaboration lean-based long chain-of-thought reasoning enhances formal theorem proving, 2025. *arXiv preprint*.
- [4] Yuji Wang et al. Matharena: Evaluating llms on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.